Python/Pytest Automation Test Sheet

Webinar 6 — execution, triage and evidence

Environment

Field	Value
Python	3.13.1
pytest	9.1.1
pyserial	3.5
DEVICE_PORT	rfc2217://localhost:4000
Firmware	BUGGY
Serial	115200

Test Summary

Metric	Value
Total tests executed	49
PASSED	42
FAILED	7
Not executed (N/E)	0
Of which existing (provided by the mentor) / own	14/35

Execution Log

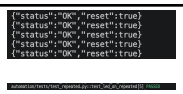
ID	Test / Input	Expected	Actual	Classification	Evidence	P/F
T-01	test_pwm_valid_boundaries [0], SET PWM 0	{"status":"OK","pwm":0}	{"status":"OK","pwm":0}	-		Pass
T-02	test_pwm_valid_boundaries [1], SET PWM 1	{"status":"OK","pwm":1}	{"status":"OK","pwm":1}	-		Pass
T-03	test_pwm_valid_boundaries [50], SET PWM 50	{"status":"OK","pwm":50}	{"status":"OK","pwm":50}	-		Pass
T-04	test_pwm_valid_boundaries [99], SET PWM 99	{"status":"OK","pwm":99}	{"status":"OK","pwm":99}	-		Pass
T-05	test_pwm_valid_boundaries [100], SET PWM 100	{"status":"OK","pwm":100}	{"status":"ERROR","code":"INVALID_VALUE"}	Firmware bug		Fail

					REQ-5. `SET PWM N` accepts every integer from 0 through 100 inclusive.	
T-06	test_pwm_invalid_boundaries[-1], SET PWM -1	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-07	test_pwm_invalid_boundaries[101], SET PWM 101	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-08	test_pwm_invalid_boundaries[999], SET PWM 999	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-09	test_led_on_off, SET LED ON -> SET LED OFF	{"status":"OK","led":true} {"status":"OK","led":false}	{"status":"OK","led":true} {"status":"OK","led":false}	-		Pass
T-10	test_reset_restores_safe_state, SET LED ON -> RESET -> STATUS	{"status":"OK","led":true} {"status":"OK","reset":true} {"status":"OK","led":false,"pwm":0,"heap":350116}	{"status":"OK","led":true} {"status":"OK","reset":true} {"status":"OK","led":true,"pwm":255,"heap":350116}	Firmware bug	 REQ-7. `RESET` restores safe output state: LED/PWM OFF/0.	Fail
T-11	test_temperature_is_valid_json, GET TEMP	`GET TEMP` → valid JSON; either valid temperature or explicit sensor error.	{"status":"OK","temperature":24.5 (closing brace "}" is missing)	Firmware bug		Fail
T-12	test_ping, PING	{"status":"OK","command":"PING"}	TimeoutError: No response for command: PING. The device responds in ~1.37 s (min 1.31 s, max 1.38 s), client timeout is 1.0 s. With timeout 3.0 s the test passes.	Test / Requirement gap		Fail (at timeout 1.0 s)

T-13	test_status, STATUS	{"status":"OK", ...}; response contains the fields heap and pwm (values are not checked)	{"status":"OK","led":true,"pwm":255,"heap":349852}	-	 The test only checks that status is OK and that heap and pwm are present. The values led:true, pwm:255 are a state leak from a previous test, because the fixture's RESET does not restore the safe state (Bug-02), so the STATUS values depend on test execution order.	Pass
T-14	test_no_response_becomes_timeout, NO_RESPONSE	TimeoutError is raised for the command NO_RESPONSE (the device does not respond).	No response from device; TimeoutError was raised and caught by pytest.raises, as expected	-		Pass
T-15	test_pwm_invalid_values[20.1]	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-16	test_pwm_invalid_values[value]	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-17	test_pwm_invalid_values[?]	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-18	test_pwm_invalid_values[20 60]	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-19	test_pwm_invalid_values 🥲 (\U0001fae1)	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-20	test_pwm_invalid_values[90 60] (value6)	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-21	test_pwm_invalid_values[0x10]	{"status":"ERROR","code":	{"status":"ERROR","code":	-		Pass

		"INVALID_VALUE"}	"INVALID_VALUE"}			
T-22	test_pwm_huge_values[2147483647]	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-23	test_pwm_huge_values[2147483648]	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-24	test_pwm_huge_values[4294967296]	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-25	test_pwm_huge_values[8882220101010101010101010023023827372377237273727372737273727372737273727372737237273]	{"status":"ERROR","code":"INVALID_VALUE"}	{"status":"ERROR","code":"INVALID_VALUE"}	-		Pass
T-26	test_invalid_pwm[SET PWM]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-27	test_invalid_pwm[SETPWM80]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-28	test_invalid_pwm[SET PWM]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-29	test_invalid_pwm[set pwm 20	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-30	test_invalid_led[SET LED]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-31	test_invalid_led[SET LED 0]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-32	test_invalid_led[SET LED 40]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-33	test_invalid_led[SET LED ONOFF]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-34	test_invalid_led[set led on]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass

T-35	test_invalid_commands[ping]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-36	test_invalid_commands[status]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-37	test_invalid_commands[GET TEMP 50]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-38	test_invalid_commands[reset]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-39	test_invalid_commands[UNKNOWN_COMMAND]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-40	test_invalid_commands[ ]	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	{"status":"ERROR","code":"UNKNOWN_COMMAND"}	-		Pass
T-41	test_led_on_repeated[2]	{"status":"OK","led":true} on every repeat	{"status":"OK","led":true} {"status":"OK","led":true}	-		Pass
T-42	test_led_on_repeated[5]	{"status":"OK","led":true} on every repeat	{"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true}	-		Pass
T-43	test_led_on_repeated[10]	{"status":"OK","led":true} on every repeat	{"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true} {"status":"OK","led":true}	-		Pass

T-44	test_reset_repeated[2]	{"status":"OK","reset":true} on every repeat	{"status":"OK","reset":true} {"status":"OK","reset":true}	-		Pass
T-45	test_reset_repeated[5]	{"status":"OK","reset":true} on every repeat	{"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true}	-		Pass
T-46	test_reset_repeated[10]	{"status":"OK","reset":true} on every repeat	{"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true} {"status":"OK","reset":true}	-		Pass
T-47	test_temp_repeated[2]	Valid JSON {"status":"OK","temperature":<-40..80>} on every repeat	{"status":"OK","temperature":24.5 (closing brace } is missing	Firmware bug		Fail
T-48	test_temp_repeated[5]	Valid JSON {"status":"OK","temperature":<-40..80>} on every repeat	{"status":"OK","temperature":24.5 (closing brace } is missing	Firmware bug		Fail
T-49	test_temp_repeated[10]	Valid JSON {"status":"OK","temperature":<-40..80>} on every repeat	{"status":"OK","temperature":24.5 (closing brace } is missing	Firmware bug		Fail

FAIL Classification

Category	Check	Result

Test	Assert/oracle correct?	Checked for each FAIL. For test_pwm_valid_boundaries[100], test_reset_restores_safe_state and test_temperature_is_valid_json the asserts match the requirements, so the tests are correct. test_ping: client timeout (1.0 s) is shorter than the measured PING response time (~1.37 s); the timeout is not backed by a requirement.
Environment	DEVICE_PORT/dependencies/port ownership?	Port works (connection via RFC 2217 to Wokwi); 42 of 49 tests pass, so the connection is stable. Dependencies installed in .venv, no other program holding the port. No environment issue found.
Communication	Timeout/transport/raw bytes?	No transport issue: the device responds consistently, only slower than the client timeout.
Requirement	Expected explicitly supported?	Yes for PWM (REQ-5: 0..100 inclusive, REQ-6: outside range gives INVALID_VALUE) and RESET (restores LED/PWM OFF/0). PING and valid JSON are also covered by the requirements.
Firmware	Repeatable mismatch with requirement?	3 defects confirmed: SET PWM 100 returns INVALID_VALUE (REQ-5); RESET leaves led: true, pwm: 255; GET TEMP returns JSON without closing } (confirmed by 4 tests: 1 provided, 3 own). All reproduce on every run.

Bug Report

Bug-01

Title: SET PWM 100 returns INVALID_VALUE, although values from 0 to 100 inclusive should be allowed.

Requirement: REQ-5. `SET PWM N` accepts every integer from 0 through 100 inclusive.

Environment: MacBook Pro, macOS Tahoe 26.5.1, VS Code 1.138.0, Wokwi Simulator, ESP32 DevKit v1

Preconditions: Open VS Code, launch Wokwi project, set up the environment for running automated tests.

Test/Steps:

1. Run test_pwm_valid_boundaries[100] test

Steps for manual testing:

1. Type “SET PWM 100”
2. Press the Enter button.

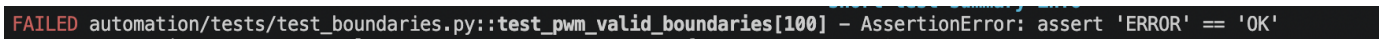
Expected: {"status":"OK","pwm":100}

Actual: INVALID_VALUE returns

Raw Serial Evidence: {"status":"ERROR","code":"INVALID_VALUE"}



Pytest Evidence: FAILED automation/tests/test_boundaries.py::test_pwm_valid_boundaries[100] - AssertionError: assert 'ERROR' == 'OK'



Severity: Major

Reproducibility: 5/5 Reproducible

Bug-02

Title: After RESET the device does not return to a safe state (led: true, pwm: 255 instead of OFF/0).

Requirement: REQ-7. `RESET` restores safe output state: LED/PWM OFF/0.

Environment: MacBook Pro, macOS Tahoe 26.5.1, VS Code 1.138.0, Wokwi Simulator, ESP32 DevKit v1

Preconditions: Open VS Code, launch Wokwi project, set up the environment for running automated tests.

Test/Steps:

1. Run test_reset_restores_safe_state test

Steps for manual testing:

1. Type “SET LED ON”
2. Type “RESET”
3. Type “STATUS”

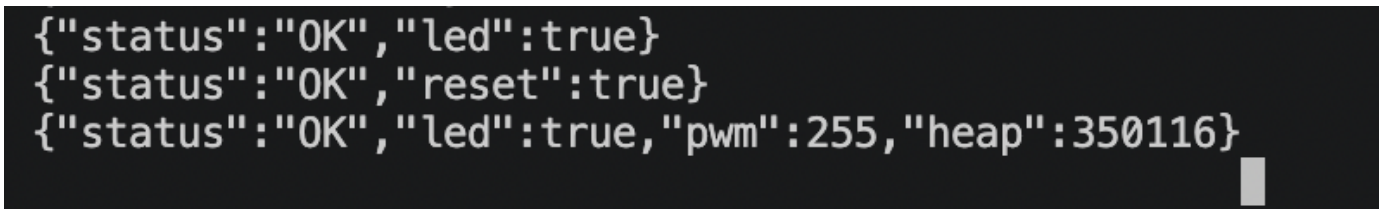
Expected: {"status":"OK","led":false,"pwm":0,"heap":350116}

Actual: led: true, pwm: 255

Raw Serial Evidence: {"status":"OK","led":true}

{"status":"OK","reset":true}

{"status":"OK","led":true,"pwm":255,"heap":350116}



Pytest Evidence: FAILED automation/tests/test_relay.py::test_reset_restores_safe_state - assert 255 == 0



Severity: Critical

Reproducibility: 5/5 Reproducible

Bug-03

Title: GET TEMP response is invalid JSON (the closing brace } is missing)

Requirement: REQ-3. `GET TEMP` → valid JSON; either valid temperature or explicit sensor error.

Environment: MacBook Pro, macOS Tahoe 26.5.1, VS Code 1.138.0, Wokwi Simulator, ESP32 DevKit v1

Preconditions: Open VS Code, launch Wokwi project, set up the environment for running automated tests.

Test/Steps:

1. Run test_temperature_is_valid_json test

Steps for manual testing:

1. Type “GET TEMP”
2. Press Enter

Expected: `GET TEMP` → valid JSON; either valid temperature or explicit sensor error.

Actual: {"status":"OK","temperature":24.5 - closing brace "}" is missing

Raw Serial Evidence: {"status":"OK","temperature":24.5



Pytest Evidence: FAILED automation/tests/test_sensor.py::test_temperature_is_valid_json - AssertionError: Invalid JSON from device: '{"status":"OK","temperature":24.5'



Severity: Major

Reproducibility: 5/5 Reproducible

Conclusions

Boundary, negative, and repeated-action parametrized tests were added to check invalid and extreme inputs.
49 tests were run; 7 FAILED.

Firmware defects:
1. SET PWM 100 returns INVALID_VALUE, although requirement REQ-5 allows values 0..100 inclusive.
2. After RESET the device does not return to a safe state (led: true, pwm: 255 instead of OFF/0).
3. GET TEMP response is invalid JSON: the closing brace } is missing. This defect is confirmed by 4 tests (1 provided and 3 own test_temp_repeated tests), so it reproduces consistently.

test_ping - measurements showed that the device responds consistently in ~1.37 s (min 1.31 s, max 1.38 s), while the client waits only 1.0 s. This is a gap between the test and the requirements (the timeout is not backed by a requirement), not a transport failure or a firmware defect. With the timeout increased to 3.0 s the test passes. Recommendation: define the expected PING response time in the requirements and set the test timeout accordingly.

Timeout behaviour works correctly: test_no_response_becomes_timeout passes, the test is written correctly, and nothing needs to be added to it.

test_status passes, but its led/pwm values are inherited from earlier tests (state leak caused by the RESET defect); the provided test does not verify these values.

STATUS reports PWM on a 0..255 scale (e.g. SET PWM 50 → STATUS shows 127), while SET PWM responses use 0..100.

The tests test_pwm_invalid_values and test_pwm_huge_values were added to the existing test_boundaries.py file. Also added two new files: test_commands.py and test_repeated.py.

conftest.py was updated to reset the device before and after each test and ensure the port is always closed. Previously, the fixture only performed cleanup after the test. A precondition was added to execute RESET and verify status == "OK". The yield was moved inside a try block, and nested try/finally blocks ensure cleanup and port closure even if the test fails.

All other custom tests passed.